Network Port Scanner

Guide d'apprentissage du code

Portfolio Cybersécurité
Architecture, bibliothèques Python et concepts réseau
Objectif : comprendre un scanner de ports comme Nmap, en Python pur

Projet v1.0 | Python 3.10+ | Bibliothèque standard uniquement

1. Introduction et objectifs pédagogiques

Ce document explique en détail le projet Network Port Scanner : un programme Python qui teste les ports ouverts d'une adresse IP, identifie les services exposés et produit un rapport de risque. Il s'inscrit dans une démarche d'apprentissage pour une carrière en cybersécurité, notamment au Japon.

Objectifs d'apprentissage

- Comprendre le scan TCP (handshake 3-way) et UDP.
- Uti

Prérequis

- Python : variables, fonctions, classes, exceptions, dictionnaires.
- Ré

Avertissement légal

Ne scannez jamais un système sans autorisation écrite. Au Japon, la Unauthorized Computer Access Law (Act No. 128, 1999) interdit l'accès non autorisé. Cibles légales : scanme.nmap.org, 127.0.0.1, votre réseau local avec permission.

2. Architecture du projet

Le code est organisé en trois modules avec des responsabilités distinctes. Cette séparation facilite la maintenance, les tests et la lecture pédagogique.

```
ScannerPortReseau/
```

Flux d'exécution

- L'utilisateur lance scanner.py (CLI ou interactif).

- res

3. Choix des bibliothèques Python

Le scanner n'utilise aucune dépendance externe (pip). Tous les modules proviennent de la bibliothèque standard (stdlib). Ce choix est pédagogique et professionnel pour un premier projet portfolio.

Pourquoi la stdlib ?

- Installation simple : python scanner.py suffit.

- Co

Modules utilisés et alternatives

Module	Rôle dans le projet	Alternative externe
socket	Connexions TCP/UDP bas niveau	scapy, python-nmap
threading	Parallélisme I/O-bound	asyncio, concurrent.futures
queue.Queue	Distribution des ports aux workers	multiprocessing
argparse	Interface en ligne de commande	click, typer
json	Export structuré des rapports	pydantic, orjson
re	Extraction version depuis bannières	-
datetime	Durée et horodatage du scan	-
os	Création du dossier reports/	pathlib
sys	Arguments CLI, codes de sortie	-

socket : fondation du scan

socket expose l'API Berkeley Sockets. connect_ex() réalise un scan de type TCP connect (équivalent Nmap -sT). Il ne nécessite pas de droits administrateur, contrairement au SYN scan (-sS) qui utilise des raw sockets.

threading vs asyncio

Le scan attend surtout des réponses réseau (I/O-bound). Les threads libèrent le GIL pendant les attentes socket. threading est plus simple à enseigner qu'asyncio pour un premier projet réseau.

argparse

argparse valide les types, génère --help et définit des valeurs par défaut. Exemple : python scanner.py -t 192.168.1.1 --ports 1-1024 --threads 100

4. Explication de scanner.py

4.1 COMMON_PORTS et RISKY_PORTS

COMMON_PORTS mappe ~20 ports courants (22 SSH, 80 HTTP, 443 HTTPS...). RISKY_PORTS liste les ports à risque élevé si ouverts (Telnet 23, RDP 3389, SMB 445, Redis 6379, MongoDB 27017...).

4.2 scan_tcp_port()

Algorithme :

- Créer socket(AF_INET, SOCK_STREAM) pour TCP/IPv4.

- Dé

4.3 scan_udp_port()

Envoi d'un paquet UDP vide via sendto(). Réponse UDP = open. Timeout = open|filtered (ambigu). ConnectionResetError = closed (ICMP unreachable).

4.4 Classe PortScanner

Implémente le pattern Worker Pool : Queue + N threads. threading.Lock() protège la liste open_ports (évite race condition). scan() lance les workers et join() attend la fin. scan_common_ports() scanne uniquement COMMON_PORTS.

4.5 Interface CLI

- build_parser() : définit -t, -p, --threads, --timeout, -o...

- pa

5. Explication de utils.py

5.1 DNS

resolve_hostname() : gethostbyname() retourne une IPv4. reverse_dns_lookup() : gethostbyaddr() pour OSINT (IP vers nom).

5.2 Banner grabbing

grab_banner() lit les octets renvoyés par le service. BANNER_PROBES contient des requêtes par port (ex: HEAD / HTTP/1.0 pour port 80). Le texte est nettoyé (première ligne, caractères imprimables, max 200 chars).

5.3 Analyse CVE

analyze_banner() applique des regex (VERSION_PATTERNS) pour extraire logiciel et version, puis compare à KNOWN_VULNERABILITIES. En production : API NVD ou searchsploit. Alertes contextuelles pour Telnet, FTP et HTTP non chiffré.

5.4 Utilitaires

- get_service_name() : lookup local ou getservbyport().

- is_

6. Explication de report.py

6.1 Classe ScanReport

Agrège les résultats du scan et produit un livrable d'audit. Enrichit les données avec reverse DNS et détection réseau privé.

6.2 Méthodes

- _calculate_risk_summary() : comptage HIGH/MEDIUM/LOW.

- _g

6.3 Sections du rapport

- Informations cible (IP, hostname, réseau privé).

- Inf

7. Concepts réseau et sécurité

7.1 Ports

0-1023 : well-known. 1024-49151 : registered. 49152-65535 : dynamiques.

7.2 Handshake TCP

SYN -> SYN-ACK -> ACK. Notre scanner utilise un TCP connect scan, visible dans les logs serveur.

7.3 Comparaison Nmap

Fonctionnalité	Ce projet	Nmap
TCP connect	**Oui**	**Oui (-sT)**
SYN stealth	**Non**	**Oui (-sS, root)**
UDP avancé	**Basique**	**Oui (-sU)**
Détection OS	**Non**	**Oui (-O)**
Scripts NSE	**Non**	**Oui**

7.4 Surface d'attaque

Chaque port ouvert augmente la surface d'attaque. Appliquer le principe du moindre privilège : fermer tout service inutile.

8. Exercices et prolongements

Débutant

- Scanner scanme.nmap.org en CLI et interactif.
- Co

Intermédiaire

- Implémenter --no-banner dans scan_tcp_port().
- Ajo

Avancé

- Réécrire avec asyncio.
- Int

Ressources Japon

- RISS (Registered Information Security Specialist) - certification JP.
- No

9. Glossaire

Banner
Texte envoyé par un service indiquant logiciel/version.

CVE
Identifiant public d'une vulnérabilité connue.

DNS
Traduction nom de domaine vers adresse IP.

Fingerprinting
Identification précise d'un logiciel via signaux réseau.

GIL
Verrou global Python limitant le parallélisme CPU.

I/O-bound
Tâche limitée par le réseau/disque, pas le processeur.

OSINT
Renseignement open source (DNS inverse, bannières...).

Pentest
Test d'intrusion autorisé.

Race condition
Bug d'accès concurrent non synchronisé.

Worker Pool
File de tâches + workers parallèles.

TCP connect scan
Scan via connexion TCP complète (connect_ex).

Filtered port
Port bloqué par firewall sans réponse claire.

Conclusion

En maîtrisant ce code, vous comprenez les briques essentielles d'un outil de reconnaissance réseau : sockets, parallélisme, DNS, bannières et documentation d'audit. C'est une base solide pour évoluer vers des outils professionnels et pour valoriser votre portfolio auprès d'employeurs en cybersécurité, au Japon et à l'international.